The Journal of Supercomputing (2025) 81:1010
https://doi.org/10.1007/s11227-025-07487-1

RESEARCH

UCP: a unified framework for code generation with pseudocode-based multi-task learning and reinforcement alignment

Yongjun Wen[1,2] · Zhihao Cui[1,2] · Yihao Liu[3] · Zhao Zhang[1,2] · Jiake Zhou[1,2] · Lijun Tang[1,2]

Accepted: 20 May 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Pre-trained large language models (LLMs) have been widely applied to natural language-based code generation. However, because code generation tasks are highly sensitive to structured information and exhibit diverse logical forms, directly adapting methods optimized for natural language tasks to code generation often proves less effective for open-source models. Conventional fine-tuning is generally limited to single, specific scenarios, while the standard Chain-of-Thought (CoT) approach differs from code in both expression and logical structure. Consequently, existing methods lack code-specific optimization strategies for downstream tasks. To address these issues, this paper leverages the unique logical interpretation and structured information of programming languages, specifically, pseudocode reflecting abstract syntax features, to construct a unified code generation framework called UCP, which integrates multi-task learning and enhanced alignment. The goal is to improve the code generation capabilities of open-source LLMs. By designing tasks that generate code from pseudocode, we build a multi-task dataset to jointly fine-tune LLMs and apply reinforcement learning(RL), optimizing a multi-task loss function to balance data distribution and task difficulty. We also establish consistent learning objectives at each stage through task-aligned training. Furthermore, focusing on code completion, we construct a multi-granularity code completion dataset—encompassing function-level, line-level, and token-level completions—for use in the RL phase. At the inference stage, we adopt a cyclic self-filling inference mechanism aligned with the completion task. We also design fill-in-the-blank tasks during pre-training to ensure alignment between the training and inference stages. The Qwen2.5-Coder-7B-Instruct model was post-trained using our proposed UCP, resulting in a HumanEval pass@1 metric improvement from 84.15 to 87.80% and a LiveCodeBench pass@1 metric increase from 25.4 to 27.2%, both surpassing other mainstream open-source models with similar parameter scales. By exposing intermediate pseudocode, UCP

Extended author information available on the last page of the article

Published online: 10 June 2025

enhances interpretability and can benefit AI-powered IDEs (e.g., Cursor), improving developer trust and real-world usability.

Keywords Pre-trained large language models · Pseudocode · Code generation · Reinforcement learning · Task alignment · Multi-task

1 Introduction

Since OpenAI's GPT series achieved breakthroughs in the NLP domain, ChatGPT and GPT-4 [1] have driven large-scale AI models to explore broader fields, making code generation an increasingly important application scenario for large language models (LLMs). Context-based code models (e.g., CodeBERT [2], CodeT5 [3], AlphaCode [4]) and generative decoder architecture models (e.g., Codex [5], SantaCoder [6], Code Llama [7]) have furthered the development of AI-assisted programming, giving rise to AI programming tools such as Github Copilot, Tabnine, Cursor, and Trae. However, general-purpose LLMs still face the following challenges when adapting to downstream code-related tasks:

1. **Limited fine-tuning efficiency and generalization.** Although LoRA (Low-Rank Adaptation) [8] and QLoRA [9] reduce the training cost for efficient fine-tuning across various downstream tasks by employing low-rank adaptation, standard fine-tuning typically relies on resource-intensive training within a single scenario. As a result, LLMs become tailored to specific tasks or contexts, making it difficult to satisfy the complex requirements of code generation. Multi-task joint fine-tuning can enhance model productivity by leveraging knowledge sharing among tasks; however, its optimization strategy must address the imbalance is constrained by data distribution and task difficulty.
2. **Lack of logical consistency and structural information.** Retrieval-Augmented Generation (RAG) [10] relies on external information to improve generation accuracy, but its performance is constrained by data quality, and the logic integration of complex code segments remains difficult. Chain-of-Thought (CoT) [11, 12] guides code generation through multi-step reasoning, yet intermediate steps can introduce noise. Therefore, it is essential to establish a "structured intermediate representation" (e.g., pseudocode) between natural language and programming languages, so as to bridge semantic gaps and ensure logical consistency in the generated code.

Pseudocode, as an abstract syntactic feature, can represent both the structured information and algorithmic logic of code. Its advantage lies in its independence from the syntax of any specific programming language while omitting low-level implementation details. Unlike code comments, pseudocode has syntactic specificity, thereby expressing that preserve the abstraction of code structure and multiple programming languages, constructing a semantic bridge between natural language and multiple programming languages.

In this paper, we design pseudocode-related task templates and leverage models such as GPT-4 and GLM4-plus [13] to construct various pseudocode datasets related to code generation. These datasets serve as the basis for multi-task joint fine-tuning and RL, enabling us to explore the performance of code models in achieving multi-task alignment across different stages. During the multi-task joint fine-tuning stage, we build datasets for six complementary tasks associated with code generation, including natural language to code (Text2Code), Text2Code tasks for multi-programming languages (TextMultilingual Code), from pseudocode to multi-programming language generation (Question → Pseudocode → Code), code to pseudocode (Code2Pseudocode), unit testing (Unit Test), and code completion (Code Completion). A multi-task joint fine-tuning loss function is employed to optimize the balance among these tasks. In the RL stage, we construct preference datasets for five tasks excluding outfit testing) and further optimize the code model using SimPO [14] reinforcement learning, under the premise of achieving multi-task alignment with the multi-task joint fine-tuning. Notably, in constructing the dataset for the question→ pseudocode→multilingual code generation task we draw inspiration from the Chain-of-Thought (CoT) paradigm by prioritizing the generation of a pseudocode explanation before generating code. The pseudocode thus serves as a guided analysis of the problem description, leading us to refer to this task as the PoT (Pseudocode-of-Thought) task. For models such as Qwen [15] that incorporate the Fill-In-the-Middle (FIM) [16] task during the pre-training phase, a cyclic Self-Infilling [17] mechanism is employed during inference to achieve consistent alignment among the pre-training FIM task, multi-task joint fine-tuning, and the code completion task in RL.

Using the proposed method, we applied further training parameter scales. The main contributions of this paper can be briefly summarized as model Qwen2.5-Coder-7B [18]. The single attempt code generation pass rate was significantly improved compared to its baseline model. Moreover, tests on the HumanEval [5], MBPP [19], and the latest LiveCodeBench [20] benchmarks to yielded scores of 87.80, 67.0, and 27.2, respectively, both surpassing those of current mainstream open-source large models and fine-tuned models with similar parameter scales. Beyond benchmarks, our pseudocode-centric design offers practical value: by exposing intermediate pseudocode, it makes the model's reasoning more transparent. This transparency can directly benefit AI-powered IDEs (such as the Cursor code editor) by providing interpretable hints and intermediate verification during development, thereby improving developer trust and usability in real programming workflows [21]. The main contributions of this paper can be briefly summarized as follows:

- We propose the PoT (Pseudocode-of-Thought) concept for code generation, which leverages pseudocode as an abstract syntactic feature. Based on this idea, we constructed a supervised fine-tuning (SFT) dataset[1] for the multi-task joint fine-tuning stage and a preference dataset for SimPO reinforcement learning. Additionally, we designed multiple tasks for both stages, including Text2Code,

[1] https://huggingface.co/datasets/Cuizhihao10/UCP

Text2Multilingual Code, PoT, Code2Pseudocode, Unit Test, and Code Completion tasks. The code completion dataset for the reinforcement learning stage is constructed at multiple granularities.

- We designed an adaptive-weighted multi-task joint fine-tuning loss function to address the balancing issues inherent in multi-task fine-tuning. Unlike typical multi-task fine-tuning on mixed datasets, our approach calculates the loss for each task individually to optimize single-task performance and then aggregates these losses with appropriate weights. The weight allocation for individual task losses is adaptively optimized and adjusted during training to resolve issues of data imbalance and task difficulty variance.
- We established a post-training workflow framework guided by pseudocode to enhance the code generation capability of open-source pre-trained large language models. This framework achieves multi-task alignment during both the multi-task joint fine-tuning and reinforcement learning stages. For the code completion task, it ensures consistent alignment from pre-training through fine-tuning, reinforcement learning, and inference. Our experimental results demonstrate that this framework can further improve the code capabilities of large code models as well as instruction-tuned models.

2 Related work

2.1 Code generation

In natural language processing (NLP), code generation tasks can be regarded as either generating code based on natural language descriptions or providing code completion suggestions based on the context of the code currently being written by the user. Code ability is a key criterion for evaluating both general-purpose large language models (LLMs) and code-specific models on code-related tasks. Currently, the most widely used benchmark for code generation is HumanEval, and models are generally categorized into closed-source and open-source groups. Among closed-source models, systems such as LaMDA [22], PaLM2 [23], ChatGPT, and GPT-4 have demonstrated outstanding performance. Notably, GPT-4 and its current iterative product, GPT-4o [24], have achieved a peak score of 92.7 on HumanEval. In contrast, most open-source models adopt a decoder-only architecture. For instance, Pangu-Coder [25] employs both masked language modeling (MLM) and a code-specific masked CLM during training. Zan [26] proposed continuous training on a sketch, whereby the model first learns to generate a program draft and subsequently writes the actual code. In the pre-training phase, Encoder [27], SantaCoder, and StarCoder [28] incorporate an FIM task, which essentially corresponds to span corruption—a technique commonly utilized in decoder—only rule extract dataset from WizardLCoder [29] is derived from StarCoder through single-task fine-tuning, while CodeT5+ [30] is fine-tuned on a diverse set of instruction datasets. MFTCoder [31] separates multiple tasks and adaptively adjusts training weights using fine-grained loss functions to achieve balance among tasks. Pangu-Coder 2 [32] leverages the Evol-Insuct dataset from

WizardLM [33] to generate 68,000 instruction samples from 20,000 code snippets, introducing RL via rank responses (RRTE) to align testing outcomes with teacher feedback. CodeRL [34] defines four recommended levels—namely, compile errors, runtime errors, unit test failures, and passes—and employs token-level rewards provided by a critic model to train an actor model(an extended version of CodeT5) using RL with human feedback(RLHF). Similarly, CompCoder [35] and PPOCoder [36] are trained using proximal policy optimization (PPO) [37] on CodeGPT and CodeT5, respectively. RLTF [38] proposes providing fine-grained equipment of test cases passed. The aforementioned work on code tasks largely neglected the intrinsic characteristics of programming languages, which may lead to ambiguities in interpreting different types of text. In contrast, this reflects an intrinsic characteristic of programming languages, which can introduce ambiguities when interpreting different forms of textual input. Rather than focusing on direct execution or behavior, this work emphasizes the interpretive aspect shared between natural and programming languages. The paper investigates their structural and semantic commonalities, with the goal of advancing research in code generation and related tasks.

2.2 Abstract syntactic features

Abstract syntactic features refer to high-level, structured characteristics extracted from programs or code. These features are typically based on the syntactic structure rather than the specific implementation details, capturing both the syntactic and semantic information of the code to facilitate code analysis, comprehension, and model learning. In the context of multi-task learning (MTL), where code generation tasks span multiple programming languages, each language exhibits distinct syntax, logic, and structural differences compared to natural language. Consequently, the intermediate representation bridging natural language and programming languages is of paramount importance in code-related tasks.

GraphCodeBERT [39] extracts structural information from source code in the form of a data flow graph (DFG) and utilizes both the source code text and the DFG to train a model that predicts the relationships between nodes. In contrast, TreeBERT [40] pioneers the incorporation of Abstract Syntax Trees (ASTs) into a Transformer-based pre-training and fine-tuning framework, employing Tree MLM and node order prediction for its pre-training process. While both data flow graphs and abstract syntax trees are valuable for understanding program semantics, each has its limitations. Data flow graphs primarily focus on the flow of data and tend to overlook control flow elements (such as conditional branches and loops), which can result in the loss of critical information when processing complex program logic. Conversely, abstract syntax trees concentrate on the syntactic structure of code rather than its semantic content, failing to directly express variable dependencies, control flow, or data flow. This limitation hampers the precise understanding of program logic during code generation. Therefore, establishing an effective connection between programming languages and natural language remains an open challenge.

This limitation hampers the precise understanding of program logic during code generation. Therefore, establishing an effective connection between programming languages and natural language remains an open challenge.

2.3 Task alignment and phase alignment

Task alignment and phase alignment refer to maintaining overall consistency between the target tasks and optimization strategies throughout the model training, fine-tuning, and inference processes. This consistency enables the model to effectively leverage the representations learned at each stage. A common method for evaluating the degree of alignment between tasks is through text alignment, thereby enabling smaller-scale models to generalize across various downstream tasks [41]. RL approaches such as DPO [42] adopt a policy-based framework that diminishes reliance on explicit value functions and maximizes the probability distribution aligned with human preferences, thus achieving congruence with human reasoning. Moreover, a consistent task structure facilitates the smoother integration of human feedback and intrinsic motivations, ensuring that LLMs generalize better and exhibit behaviors more in line with user expectations. However, these methods have yet to achieve full-process alignment, and there remains significant room for improvement in maintaining consistency across task distribution and task structure.

2.4 Commercial code assistants

Recent years have seen the rapid adoption of commercial AI-powered code assistants such as GitHub Copilot, Cursor, and Tabnine, which integrate large language models (LLMs) into mainstream development environments. These systems offer in-line code completion, multi-line generation, and natural language interaction, directly within IDEs such as VS Code, JetBrains, and Visual Studio. Compared to direct LLM APIs, they provide a more practical user experience by automatically managing prompt context, maintaining session history, and adapting to project-specific semantics.

GitHub Copilot, built on Codex and later GPT-based models, delivers contextual code suggestions and conversational assistance via "Copilot Chat" enabling developers to ask natural language queries about their codebase or documentation [43]. Benchmarks such as HumanEval show that Copilot achieves competitive synthesis accuracy, while user studies report significant productivity gains [44]. Cursor, a fork of VS Code with built-in LLM capabilities, goes further by indexing the entire codebase and enabling file-aware operations such as refactoring and multi-file edits. Tabnine focuses on privacy-preserving completions, offering models that can be fine-tuned on organizational code to reflect internal patterns and APIs.

While these tools excel at accelerating routine development tasks and improving workflow efficiency, they generally treat model outputs as black boxes, returning final code suggestions without exposing intermediate reasoning or structural planning. However, these tools lack the ability to incrementally exhibit their thought process during the code generation process, failing to provide the necessary

transparency for engineering phases. This transparency is particularly important for complex or high-stakes scenarios [45] (e.g., algorithm design, code correction, or debugging), where understanding why a solution works is as important as the solution itself.

3 Methods

3.1 UCP framework

To further explore the performance of LLMs in specific tasks within code generation, this paper proposes the UCP framework shown in Fig. 1 The framework consists of six components:

① Multi-task Fine-tuning Dataset Construction: The pseudocode-related task dataset uses general pseudocode definitions as prompts. By invoking code LLMs, we obtain corresponding pseudocode solutions for problems and combine them with related questions or code snippets. Other task datasets are constructed by filtering and combining various open-source datasets through standard code LLM invocations.

② Multi-task Design: During the multi-task joint fine-tuning phase, it includes six supervised fine-tuning (SFT) tasks as shown in the figure. The RL phase involves future preference tasks, excluding the Unit Test.

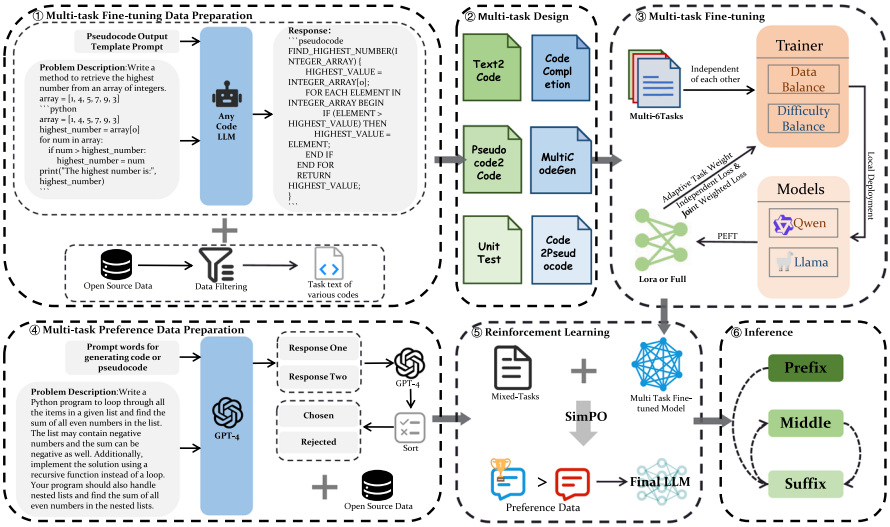

Fig. 1 UCP overall framework. The entire framework is divided into six steps across three phases. After completing the relevant tasks, a multi-task fine-tuning dataset and a RL preference dataset are constructed according to the requirements. The pretrained large language model then goes through the multi-task joint fine-tuning phase and the RL phase to obtain the final aligned model. Finally, during the inference stage, the three-phase alignment is achieved

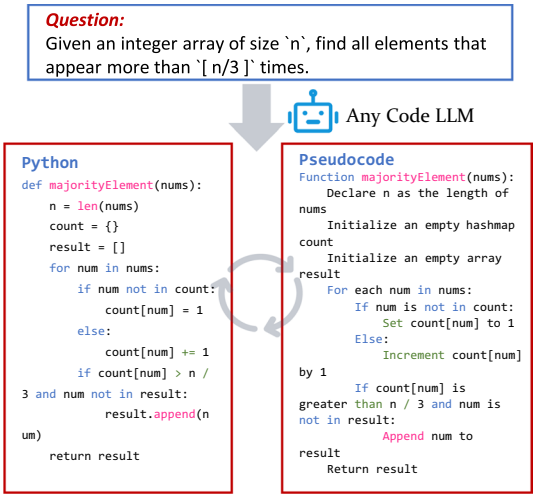

Fig. 2 Pseudocode generation example. The generated pseudocode should have a clear structure, direct variable declaration and initialization, clear operation and algorithm logic. Functions should return results in a reasonable way. Function names and variable names should reflect their functions or functions, avoid syntax details of specific programming languages, and use natural language to describe algorithm logic

③ Multi-task Joint Fine-tuning: This phase conducts multi-task training using six datasets corresponding to six tasks. A joint loss function monitors training effectiveness for each task while optimizing data balance and task difficulty balance.

④ Multi-task Preference Dataset Construction: We generate multiple answers to problems, which are then used as evaluation models to rank data preferences. This creates LLM-based preference data generation. Combined with filtered open-source preference datasets from other sports, we obtain a comprehensive hybrid multi-task preference dataset.

⑤ SimPO-based RL: Natural language metabolites used for RL to further train the model after multi-task joint fine-tuning.

⑥ Self-Infilling Inference: Implements inference using models obtained after RL. By incorporating pseudocode as abstract syntactic features of LLM training titles, the model needs to enhance the framework to enhance LLMs' understanding of natural language-to-code conversion, making them adaptable to diverse subtasks and new application scenarios.

With the overarching goal of improving LLMs' comprehensive code capabilities, the model needs to enhance performance in specific subtasks like natural language-to-code generation and code completion. However, not all subtasks contribute equally to overall capability improvement. Factors such as knowledge sharing between tasks, task relevance, and learning difficulty significantly impact final training outcomes. Intermediate representations between natural language and code using pseudocode as abstract syntactic features bridge differences between subtasks. Additionally, the framework employs task alignment across all three phases to mitigate inconsistencies between multi-stage/multi-step processes, including ensuring multi-task alignment between the first two phases and introducing blank-filling tasks corresponding to Self-Infilling inference.

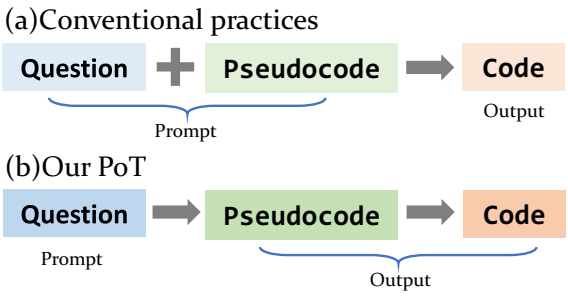

Fig. 3 PoT task design illustration

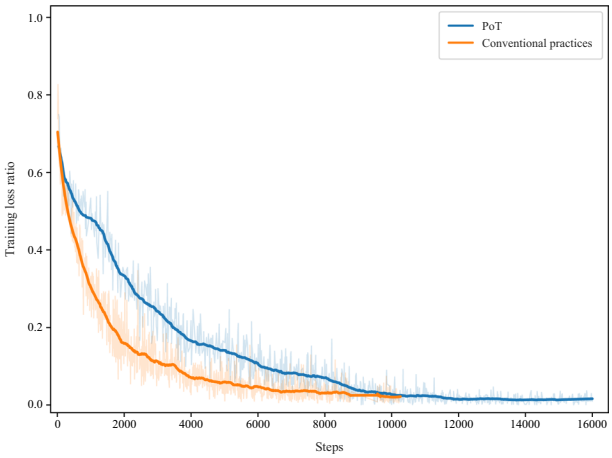

Fig. 4 Comparative LoRA fine-tuning of two pseudocode task design paradigms on Llama–3.1-8B-Instruct [47]

3.2 PoT task design and multi-task dataset construction

Pseudocode, as an abstract syntactic feature bridging natural language and code, exhibits hybrid characteristics between the two. As shown in Fig. 2, pseudocode examples generated by large code models (e.g., GPT-4) interweave natural language with program logic and structure. Unlike Abstract Syntax Trees (ASTs), which focus on code-specific syntax and statement structures while stripping natural language associations, the generated pseudocode retains contextual natural language elements (e.g., "check if the input is valid") while outlining algorithmic steps (e.g., loops, conditionals). This hybrid representation aligns with the inference patterns of prompt-tuned LLMs, enabling models to map problem requirements to code logic without over-constraining syntax. By training on such pseudocode-augmented data, the model learns to disentangle high-level intent from low-level implementation, a critical capability for generalizing across diverse code generation tasks.

The PoT task design, illustrated in Fig. 3b, bridges problem descriptions and code generation through stepwise inference. By dynamically blending contextual information from both the problem statement and pseudocode during generation, this method guides the model to deeply understand task requirements and improves code accuracy. Traditional approaches (Fig. 3a), which naively concatenate problems and pseudocode as training prompts (Fig. 4), prioritize rapid convergence but suffer from premature overfitting and destabilized multi-task balance. These issues arise because traditional methods oversimplify task difficulty and overlap excessively with subtasks like natural-language-to-code generation, creating rigid task dependencies and limiting knowledge sharing across tasks.

In multi-task joint fine-tuning and RL, the PoT paradigm is uniformly applied. Unlike Chain-of-Thought (CoT) methods that depend on multi-turn dialogue for incremental inference, PoT uses structured prompts to enforce sequential pseudocode and final code generation within a single inference pass. Key to this design is the structured prompt that enforces sequential generation: the model first produces pseudocode (capturing high-level logic) and then synthesizes executable code grounded in the pseudocode's inference. By leveraging the Transformer architecture's self-attention mechanism [46], PoT maintains long-range dependencies between problem context, pseudocode steps, and final code, ensuring alignment with both natural language intent and syntactic correctness. This approach mitigates the overfitting risks and task homogenization observed in traditional methods, while fostering robust knowledge transfer across diverse code generation subtasks.

For both the multi-task joint fine-tuning and RL phases, we established five aligned task categories: Text2Code, Text2Multilingual Code, PoT, Code2Pseudocode, and multi-granular code completion. An additional Unit Test Generation task was integrated during the joint fine-tuning phase to enhance functional validation capabilities. In the joint fine-tuning phase, tasks like Text2Code and multilingual code synthesis relied on high-quality open-source datasets, refined through similar filtering to ensure consistency and quality.

The datasets for PoT and Code2Pseudocode tasks were constructed using the pseudocode generation methodology illustrated in Fig. 1's Subfigure ①. This involved generating pseudocode logic through large code models (e.g., GPT-4) and pairing it with problem descriptions. To ensure the dataset's quality, we generate pseudocode with the same input using different language models (such as GPT-4 and GLM4-plus) through multi-model cross-validation and compare the output differences to identify potential ambiguities or errors. We sample and review the results generated by the batch to ensure compliance with pseudocode writing standards, error correction, and uniform style. To ensure the consistency of the dataset, templates and rules for PoT tasks and Code2Pseudocode tasks are defined (such as rule definitions, input/output formats, indentation structures, control structure styles, etc.), as shown in Figs. 5 and 6, and static checks are performed. Simultaneously perform forward and backward testing on the generated results, automatically generate code from pseudocode, and then convert the code back to pseudocode (reverse validation) to check if the original semantics are preserved. Code completion datasets combined outputs from code models with carefully filtered open-source data, applying constraints on code length and structural completeness.

Fig. 5 PoT task prompt templates. Prompt templates include problem descriptions, defining PoT task rule constraints, system prompts, and output example templates

```
{Problem Definition and Pseudocode Rules}
### Problem
[Natural language problem description, e.g., "Sort an array using a divide-and-conquer strategy."]

### Rules
1. **Pseudocode Generation Phase** (Phase 1):
  - Must include **function signature** (e.g., `FUNCTION quicksort(arr, low, high)`) and **parameter type declarations** (e.g., `arr: List[int], low: int`);
  - Use **structured control flow** (`IF/ELSE`, `FOR/WHILE`, `RETURN`) and **natural language operations** (e.g., `INITIALIZE pivot = arr[high]`);
  - Variables must be **explicitly declared and initialized** (e.g., `DECLARE i: int = low - 1`);
  - Avoid language-specific syntax (e.g., `{}` or semicolons); use 4-space indentation for hierarchy;
  - Add **inline comments** for critical steps (e.g., `// Partitioning phase`).

2. **Code Generation Phase** (Phase 2):
  - Code must **line-by-line mirror** pseudocode logic, preserving all variable names and operation sequences;
  - Add necessary syntax (e.g., Python `def`, Java `public static`) without altering algorithm logic;
  - Include **code comments** for key steps (e.g., `# Boyer-Moore Voting Algorithm`).

### Your Task
Please prioritize generating the corresponding pseudocode according to the pseudocode template requirements based on the above questions, and then generate the corresponding final code according to the pseudocode approach. Generate the following two sections, strictly adhering to the user-provided example (e.g., QuickSort), and require a detailed step-by-step description of the thought process and reasoning process as much as possible:
```pseudocode
[Phase 1 Pseudocode Structure]
FUNCTION [Name](Parameters):
   IF [Condition] THEN:
      [Action with variables]
   FOR [Range] DO:
      [Loop operation]
   RETURN [Result]
```

```python
[Phase 2 Code, Aligned with Pseudocode]
def function_name(params):
   # Comment explaining logical block
   variable = initial_value
   for element in collection:
      if condition:
         operation
   return result
```
```

Fig. 6 Code2Pseudocode prompt template. The prompt template includes source code blocks, pseudocode rule constraints, system prompts, and pseudocode template examples

```
{Code Abstraction Rules}
### Source Code
[Programming language code snippet, e.g.:
def reverse_linked_list(head):
   prev = None
   while head:
      next_node = head.next
      head.next = prev
      prev = head
      head = next_node
   return prev
]

### Rules
1. **Structural Preservation**:
  - Pseudocode must retain the **function signature** (e.g., `FUNCTION reverseLinkedList(head: Node)`), **control structures** (`WHILE`/`IF`), and **operation sequence** from the source code.
  - Replace language-specific syntax with **natural language keywords** (e.g., `INITIALIZE` instead of `=`, `ADVANCE` instead of pointer movement `head = head.next`).

2. **Variables and Data Flow**:
  - All variables must be **explicitly declared with types** upon first use (e.g., `DECLARE prev: Node = NULL`).
  - Annotate the initial state of data structures (e.g., `// Initialize prev as NULL to represent the previous node`).

3. **Abstraction Level Control**:
  - Remove language-specific symbols (e.g., Python `self`, Java `public`).
  - Describe low-level operations with generic terms (e.g., `SWAP` instead of memory exchange logic).

4. **Comment Enhancement**:
  - Add comments for complex logic blocks (e.g., `// Detach current node and link to previous`).
  - Label input/output constraints at function entry/exit (e.g., `// Input: head of linked list; Output: new head`).

### Your Task
Convert the source code into standardized pseudocode adhering to the following format:
```pseudocode
FUNCTION [FunctionName](ParameterList):
   [Variable Declaration & Initialization]
   [Control Structures (Loops/Conditionals)]
   [Core Operation Steps]
   RETURN [Result]
```
```

During the RL phase, the framework retained the core task alignment but excluded the Unit Test task to focus on code generation objectives. The preference dataset for the PoT and Code2Pseudocode in the RL stage is obtained using the method shown in Subfigure ④ of Fig. 1. By comparing standards clearly (specifying preference criteria for each task, such as conciseness, logicality, helpfulness, etc.), controlling

task distribution (ensuring a balanced number of task types, such as Q&A, translation, summarization, etc., covering real usage scenarios), differentiating positive and negative samples (making preference learning have clear training signals, avoiding "fuzzy preference pairs") to ensure the quality of the preference dataset, and strictly controlling the style consistency of the dataset using the prompt word template of PoT task shown in Fig. 7, we initially generated candidate pairs (prompt → responsive1/responsive2) using GPT-4/GLM4-plus and used multiple rounds of Q&A and manual review to make multiple-choice questions for the responses, which to some extent ensured the correct order of preference data with maximum accuracy, while code completion datasets were expanded to include multi-granular preference data spanning function-level, line-level, and token-level granularity. These datasets were processed using a custom partitioning method (Algorithm 1) to balance task-specific distributions and semantic alignment.

Problem Definition
[Natural language description]
Based on natural language problems, generate corresponding pseudocode ideas first, and then generate corresponding code based on the problem and pseudocode ideas. Request to generate 2 answers and sort them based on their quality, providing corresponding detailed code explanations and judgment of answer quality.
Example: "Implement a depth-first search (DFS) algorithm for tree traversal."

Two-Phase Generation Protocol

Chosen response:
Phase 1: Pseudocode Standards
```pseudocode
FUNCTION dfs(root: TreeNode, target: any) -> bool:
  // 1. Explicit Initialization
  INITIALIZE stack = [root]
  // 2. Structured Logic Flow
  WHILE stack not empty DO:
    INITIALIZE node = stack.pop()
    // 3. Natural Language Operations
    IF node.value == target THEN:
      RETURN True
    // 4. Child Node Processing
    IF node.right != null THEN:
      stack.append(node.right)
    IF node.left != null THEN:
      stack.append(node.left)
  RETURN False  // Edge case handling
```
Phase 2: Code Implementation
```python
class TreeNode:
  def __init__(self, val=0, left=None, right=None):
    self.val = val
    self.left = left
    self.right = right
def dfs(root: TreeNode, target: int) -> bool:
  # 1:1 mapping from pseudocode
  stack = [root]  # Phase 1: INITIALIZE
  while stack:    # Phase 1: WHILE loop
    node = stack.pop()
    if node.val == target:  # Direct translation
      return True
    # Reverse order for LIFO behavior
    if node.right:        # Phase 1: IF condition
      stack.append(node.right)
    if node.left:
      stack.append(node.left)
  return False  # Full coverage
```

Rejected response:
Phase 1: Pseudocode Standards
```pseudocode
FUNCTION dfs_preorder(root: TreeNode) -> List[int]:
  // Initialize traversal list and stack
  INITIALIZE result = []
  INITIALIZE stack = [root]
  // Process nodes in LIFO order
  WHILE stack is not empty DO:
    CURRENT node = stack.pop()
    APPEND node.val TO result
    // Add child nodes to stack
    IF node.right != null THEN:
      stack.push(node.right)  # ❌ Reverse child order
    IF node.left != null THEN:
      stack.push(node.left)
  RETURN result
```
Phase 2: Code Implementation
```python
class TreeNode:
  def __init__(self, val=0, left=None, right=None):
    self.val = val
    self.left = left
    self.right = right
def dfs_preorder(root: TreeNode) -> list[int]:
  if not root:
    return []
  result = []
  stack = [root]
  while stack:
    # 1. Process current node
    node = stack.pop()
    result.append(node.val)
    # 2. Push children to stack
    if node.right:  # ❌ Subtle inversion
      stack.append(node.right)
    if node.left:
      stack.append(node.left)
  return result  # Output resembles pre-order but has hidden flaws
```

Answer analysis
Correct: Properly implements pre-order DFS by pushing right child first→ left child last to stack (LIFO), ensuring Root→Left→Right traversal.
Incorrect: Reverses child order (right pushed first but processed first), creating Root→Right→Left order while misleadingly claiming "pre-order".
Key Difference: Stack insertion sequence dictates traversal logic - correct version maintains LIFO discipline for true DFS; flawed version inverts subtree processing.

Fig. 7 Preference data prompt template. The prompt template includes system prompts, problem descriptions, PoT preference data rule constraints, preference data example templates, and comparison analysis examples of 2 types of answers

Algorithm 1 Multi-granular Code Completion Dataset Construction

Input: Code snippet C, Tokens: `<deleted_line>`, `<deleted_word>`
Output: Datasets: $L_dataset$, $W_dataset$

```
for each C in dataset do
    if random() < 0.8 then
        L_del ⇐ random_int(1, 7)
        C_modified ⇐ delete_lines(C, L_del, <deleted_line>)
        chosen, rejected ⇐ generate_completions(C_modified, L_del)
        L_dataset.append((C, chosen, rejected))
    else
        line ⇐ select_random_line(C)
        W_del ⇐ random_int(1, 10)
        C_modified ⇐ delete_words(line, W_del, <deleted_word>)
        chosen, rejected ⇐ generate_completions(C_modified, W_del)
        W_dataset.append((C, chosen, rejected))
    end if
end for
Ensure_ratio(L_dataset, W_dataset, 8:2)
return L_dataset, W_dataset
```

For line-level completion data, a code snippet C is processed by randomly selecting non-initial, non-comment lines of code and replacing them with the special token `<deleted_line>`. This modified snippet serves as the input prompt. The original code is reconstructed by querying models like GLM4-plus and GPT-4, which generate multiple candidate completions. These candidates are ranked, with the best and worst outputs selected to form a paired (high-quality, low-quality) data sample. Token-level completion data follow a similar approach but replaces individual tokens (instead of full lines) with `<deleted_word>`. When processing the first line of a code snippet, the initial token is preserved to maintain structural context. The dataset balances line-level and token-level samples at an 8:2 ratio, ensuring a focus on broader code logic while retaining fine-grained synthesis capability. These are combined with filtered open-source function-level completion data (e.g., missing function bodies) to create a comprehensive multi-granular code completion dataset. This hybrid design enables models to learn both high-level code structure and detailed syntax patterns, aligning with real-world development scenarios where programmers iteratively refine code at varying granularities.

In the process of building the dataset, for the PoT task, we primarily constructed examples using four languages: Python, C++, Java, and JavaScript. Across two stages, we built approximately 200,000 samples for all task types in total, amounting to about 200 million tokens, with Python being our main focus. The sample data ratio for Python, C++, Java, JavaScript, and other languages (such as Golang, SQL, C#, Ruby, etc.) is approximately 4:1:1:1:1. The detailed information of each task dataset in the multi-tasking fine-tuning stage and RL stage is shown in Table 1. The dataset does not involve constructing or utilizing

Table 1 Training tasks at each stage and corresponding training data

Task	Desired ability	#Samples
Multi-task Joint Fine-tuning		
Text2Code	Natural language generated code(Python)	27224
Text2Multilingual Code	Natural language generates multilingual code	66862
PoT	Question to pseudocode and then to code	22048
Code2Pseudocode	Generate pseudocode from code	10000
Code Completion	Code Completion	20000
Unit Test	Unit test-case generation	13000
Total Multi-task Joint Fine-tuning	All 6 of the above	159134
Reinforcement Learning		
Text2Code	Natural language generated code(Python and Direct Preference)	7180
Text2Multilingual Code	Natural language generated code(Direct Preference)	9466
PoT	Question to pseudocode and then to code(Direct Preference)	6237
Code2Pseudocode	Generate pseudocode from code(Direct Preference)	5702
Code Completion	Multi-granular Code Completion(Direct Preference)	8403
Total Reinforcement Learning	All 5 of the above	36988
Two stages in total	All of the above	196122

abstract syntax trees (ASTs), and all intermediate representations are presented as human-readable pseudocode with natural language annotations.

3.3 Dynamic loss-balanced multi-task joint fine-tuning

As shown in Table 1, the multi-task joint fine-tuning process involves significant disparities in dataset sizes and task difficulty across the six tasks. For instance, code completion tasks have substantially larger datasets than unit test generation, while pseudocode-guided inference (PoT) is inherently more complex than straightforward code synthesis. Such imbalances can bias the model toward optimizing for high-resource or easier tasks, undermining the collaborative benefits of multi-task learning. To address this, we propose a dynamic loss-balancing strategy that adaptively adjusts task weights based on the statistical characteristics of each training batch. The core idea is to weight each task's contribution to the total loss by its effective token count—the number of non-special tokens (e.g., excluding <deleted_line>, <pad>) in the current batch. For example, a PoT task sample requiring long pseudocode generation might contain 200 effective tokens, while a partially masked code completion sample might have only 50. By dynamically scaling task weights according to Eq. (1), the model automatically assigns higher weights to tasks with fewer effective tokens per sample, ensuring balanced contributions to the total loss during gradient updates.

$$\mathcal{L}(\theta) = \min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \frac{\sum_{j=1}^{S_i} \sum_{k=1}^{T_{ij}} - \log(p_\theta(t_{ijk}))}{\sum_{j=1}^{M_i} T_{ij}} \tag{1}$$

where N is the number of multi-task categories (six in this study), S_i represents the number of samples for the i-th task, T_{ij} represents the total number of effective tokens for the j-th sample of the i-th task, and $p_\theta(t_{ijk})$ represents the prediction probability of the model for the tag t_{ijk}.

To address the task difficulty imbalance in multi-task learning, this study introduces task-level adaptive weighting [48]. By analyzing the changes in validation loss for each task, we evaluate their convergence rates. Based on both the convergence speed and task importance, the method dynamically updates task weights, directing the model to prioritize harder-to-learn tasks. This approach mitigates imbalances caused by varying task difficulties or data distribution disparities. The weight adjustment mechanism is formalized in Eq. (2), where task weights are refined iteratively to align optimization efforts with the evolving training dynamics.

$$w_i = \text{softmax}\left(\gamma \cdot \frac{\mathcal{L}_i^{\text{prev}} - \mathcal{L}_i^{\text{current}}}{\sum_{j=1}^{N} (\mathcal{L}_j^{\text{prev}} - \mathcal{L}_j^{\text{current}})} \right) \tag{2}$$

Here, w_i denotes the weight of the i-th task, and γ is a scaling factor controlling the sharpness of the weight distribution. $\mathcal{L}_i^{\text{prev}}$ and $\mathcal{L}_i^{\text{current}}$ represent the validation loss of the i-th task at the previous and current time steps, respectively. These metrics quantify the task's convergence progress over training iterations.

The combined loss that balances data and task difficulty is shown in Eq. (3). $\mathcal{L}_i$ represents the loss of the i-th task under data balance, while w_i denotes the task weight of the i-th task.

$$\mathcal{L} = \sum_{i=1}^{N} \mathcal{L}_i w_i \tag{3}$$

3.4 SimPO-based reinforcement learning for multi-task code generation optimization and alignment

After multi-task joint fine-tuning, the model still faces problems such as irregular code generation and a lack of code feedback information. To address these limitations, we integrate multi-task joint fine-tuning with reinforcement learning, leveraging the SimPO (simplified preference optimization) method to build a more robust and adaptable code generation framework. SimPO offers critical advantages over other approaches like PPO and DPO, including its reference model-free architecture, direct alignment between reward signals and generation objectives, and consistent behavior across training and inference phases. These properties enable seamless alignment with the multi-task joint fine-tuning phase while ensuring compatibility with downstream code synthesis goals. At the same time, based on the experimental

conditions of this study, its advantages in training time are relatively short and GPU memory consumption is low, which is also conducive to efficient phase-to-task alignment in the code generation process. These properties enable seamless alignment with the multi-task joint fine-tuning phase while ensuring compatibility with downstream code synthesis goals.

As shown in Eq. (4), SimPO employs the average log probability of a generated sequence as its implicit reward, normalized by sequence length to avoid favoring overly long but low-quality responses. This length normalization ensures the reward metric prioritizes concise and accurate outputs. To further enhance policy quality, SimPO introduces a target reward margin that enforces clear differentiation between high-quality (winning) and low-quality (losing) responses.

$$\mathcal{L}_{\text{SimPO}}(\pi_\theta) = -\mathbb{E}\left[\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l|x) - \gamma\right)\right] \tag{4}$$

During the construction of preference data in Sect. 3.2, we incorporated LLM-generated evaluation feedback to enhance code correctness and style scoring metrics (derived from both model outputs and open-source data) to improve code readability. To minimize catastrophic forgetting of knowledge acquired during multi-task joint fine-tuning and mitigate conflicts between tasks, the RL phase strategically controls training scale. Specifically, the RL dataset size is maintained at a 1:4 ratio relative to the fine-tuning phase, preventing over-specialization while preserving generalization. By blending multi-task datasets aligned with the five core tasks—Text2Code, Text2Multilingual Code, PoT, Code2Pseudocode, and Code Completion—the framework ensures continuous exposure to diverse task distributions during optimization. This prevents performance degradation on non-target tasks caused by single-task overfitting.

The mixed preference dataset whose examples fall into two buckets:

1. **Task fidelity:** pairs comparing outputs from different multi-task heads (e.g., two code completions for the same prompt).
2. **PoT style:** pairs comparing step-by-step pseudocode/inference answers for accuracy or format correctness under the same prompt.

We calibrated the ratio of these buckets (roughly 67% task-fidelity vs. 33% PoT) so that the final policy would both retain its raw coding accuracy and significantly improve its adherence to PoT.

In the setting of hyperparameters, simpo_gamma=5.0(γ=5.0) acts as a factor similar to a discount, amplifying the difference in advantage between preferred and non-preferred samples. A higher γ makes the algorithm place more emphasis on examples that strongly satisfy the combined objective. Meanwhile, β=10.0 serves as the KL divergence penalty coefficient, connecting the updated policy with the RL (multi-task) checkpoint prior to the update. A larger β enforces stronger regularization, preventing the policy from deviating too far while pursuing higher intrinsic rewards. The regularized update strategy, when β is set to 10.0, ensures that the KL term prevents over-optimization of the PoT objective at the expense of the original

code's accuracy. During training, we linearly decay β to gradually increase flexibility once the policy aligns with the PoT objective. Through a combination of blended data, controlled training scales, dynamic reward mechanisms, and preference-based feedback, the RL phase not only retains the foundational knowledge from multi-task fine-tuning but also refines task-specific capabilities. For instance, multilingual code generation benefits from cross-lingual consistency rewards, while pseudocode-to-code translation prioritizes logic preservation. As demonstrated in Sect. 4.2.1 experiments, this systematic integration reduces task conflicts and catastrophic forgetting. The resulting model achieves holistic improvements in code correctness and task adaptability across diverse programming scenarios.

3.5 Self-Infilling inference for task alignment optimization

In the first two stages, the alignment of code completion tasks has been achieved. In order to better achieve the alignment of tasks in the three stages and achieve further alignment with the inference stage of the large model of the autoregressive decoder architecture, the Self-Infilling method is introduced in this paper, as shown in Subfigure ⑥ in Fig. 1. The filling operation aims to fill the middle part according to the predefined prefix and suffix and generate these contexts and filling content in turn through self-supplementation. As shown in Eq. (5), further use the cyclic mechanism to update and synchronize each generated part in a cyclic manner, allowing the generation of uncertain code fragments to be postponed until the exact suffix is established, so as to better control the output sequence and effectively adapt to the nonmonotonic generation of complex code logic.

$$\begin{aligned} \text{middle}^{(n)} &\sim p(\text{middle} \mid \texttt{prefix}, \texttt{suffix}^{(n-1)}) \\ \texttt{suffix}^{(n)} &\sim p(\texttt{suffix} \mid \texttt{prefix}, \text{middle}^{(n)}) \\ y &= [\texttt{prefix};\text{middle}^{(N)};\texttt{suffix}^{(N)}] \end{aligned} \tag{5}$$

where $\text{middle}^{(n)}$ indicates that in the n-th iteration, the middle part is generated based on the prefix and the suffix of the previous iteration, and $\text{suffix}^{(n)}$ indicates that in the n-th iteration, the suffix part is generated based on the prefix and the middle part of the current iteration. The final y indicates that the final generated code is a combination of the prefix, the middle part of the last iteration, and the suffix.

In terms of optimizing inference time, the algorithm already uses an uncertainty threshold (τ) to interrupt prefix decoding and switch to suffix generation once the model is unsure. One can tune τ or implement additional stop conditions to minimize unnecessary token emissions. Similarly, the total number of loops N could be made dynamic: in many cases, the first cycle yields a high-quality draft, and further loops give diminishing returns. Halting early when improvements plateau would save repeated decoding. And instead of always running full passes, one can generate only what is needed. For example, if a generated suffix is syntactically complete (e.g., reaches a closing brace or return statement), the decoder could stop early. Techniques like nucleus or top-k sampling with strict stop tokens can avoid

overshooting the target. In effect, this is like an "early stopping" strategy on the output side.

4 Experiments and results analysis

This chapter presents experiments to validate the effectiveness of the proposed pseudocode-driven task designs and framework. We conduct ablation and comparative studies across all framework stages and tasks, focusing on three key questions: (1) the necessity of phased task alignment and the overall efficacy of the three-stage framework (§4.2), (2) the rationality of multi-task design and the impact of multi-task joint fine-tuning on enhancing LLMs' code capabilities (§4.2.1), and (3) whether pseudocode as an abstract syntactic feature improves LLMs' performance in code generation (§4.2.2).

All experiments use Llama−3.1-8B-Instruct and Qwen2.5-Coder-7B-Instruct as baselines. The training and inference environments employ single-node 8×V100 (32GB) GPUs, with FP16 mixed-precision training and FP16 half-precision inference unless otherwise specified.

4.1 Evaluation datasets

This study employs publicly available and representative code evaluation benchmarks for comparative assessment.

HumanEval, curated by OpenAI researchers, consists of 164 Python programming problems designed to evaluate code generation models. Each problem includes approximately 9.6 test cases to verify the correctness of the generated code, establishing HumanEval as a widely adopted benchmark for measuring LLMs' coding capabilities.

HumanEval+ [49] extends HumanEval with additional test cases and more complex tasks to better reflect real-world programming scenarios. This enhanced benchmark effectively identifies previously undetected errors in LLM-generated code, demonstrating its utility for rigorous evaluation.

MBPP, a crowdsourced dataset containing 1,000 Python programming problems, focuses on assessing models' proficiency in basic Python syntax and problem-solving. For this study, a subset of 500 problems (IDs 11-510) was selected to evaluate text-to-code generation performance, particularly the ability to synthesize code from natural language problem descriptions.

LiveCodeBench, a comprehensive and contamination-resistant framework, evaluates the coding capabilities of large language models. By dynamically curating fresh coding challenges from ongoing contests on three major programming platforms—LeetCode, AtCoder, and Codeforces—it ensures temporally grounded evaluation free from data leakage risks. The benchmark's latest release (v6) features 1,055 meticulously curated coding problems spanning challenges released between May 2023 and April 2025, each rigorously validated through historical version

control analysis and paired with comprehensive test cases to enable robust capability assessment.

As the principal evaluation criterion, the pass@1 metric measures the first-attempt accuracy of code generation by assessing the probability of producing correct code on the initial execution. In select experiments, pass@10 (correctness within ten attempts) is additionally reported to assess model robustness under iterative sampling. These benchmarks collectively provide a comprehensive framework for analyzing code generation accuracy, generalization, and task adaptability.

4.2 Evaluation results

Under laboratory conditions, this paper uses the framework described in this paper to conduct post-training based on two mainstream open-source models, Llama–3.1-8B-Instruct and Qwen2.5-Coder-7B-Instruct, respectively. The final model is compared with the mainstream cutting-edge models in the range of 3B to 16B in terms of code capabilities.

The evaluation results presented in Table 2 were obtained through local deployment and accelerated inference using a single-node 4×V100 GPU cluster with the

Table 2 Pass@1 evaluation results on HumanEval, MBPP and LiveCodeBench Benchmarks. All methods are evaluated using the strategy of combining greedy decoding and zero shot. Except for the closed-source model, the evaluation results of all other models are based on the same experimental conditions in this paper and are evaluated using vLLM [50] local deployment and accelerated inference

Methods	HumanEval pass@1	MBPP pass@1	LiveCodeBench pass@1
Closed-source models			
GPT-4 [1]	85.4	N/A	28.8
GPT-4o [24]	92.7	N/A	29.5
Open-source models			
Llama–3.2-3B-Instruct [47]	56.71	50.8	11.2
Qwen2-7B-Instruct [51]	78.05	53.6	15.8
Mistral-7B-Instruct-v0.3 [52]	43.90	33.6	7.6
Qwen2.5-Coder-7B-Instruct [18]	84.15	65.4	25.4
Llama–3.1-8B-Instruct [47]	66.46	57.6	15.7
Ministral-8B-Instruct-2410 [53]	74.39	N/A	17.0
Mistral-Nemo-Instruct-2407 [53]	70.73	56.4	15.6
DeepSeek-Coder-V2-Lite-Instruct [54]	77.44	71.4	22.6
DeepSeek-V3-1226 [55]	82.60	**75.4**	24.0
Ours(Llama-3.1-8B-Instruct+LoRA+SimPO)	67.70	58.8	16.9
Ours(Qwen2.5-Coder-7B-Instruct+Full+SimPO)	**87.80**	67.0	**27.2**

We respectively use Llama-3.1-8B-Instruct and Qwen2.5-Coder-7B-Instruct as baseline models, perform LoRA and full-parameter multitask fine-tuning, and further conduct SimPO reinforcement learning to obtain our two final models. The highest performance results of open-source models are highlighted in bold

vLLM framework. The Llama−3.1-8B-Instruct+LoRA+SimPO model, developed through LoRA fine-tuning, and the Qwen2.5-Coder-7B-Instruct+Full+SimPO model, trained via full-parameter fine-tuning, were both optimized using DeepSpeed+ZeRO3 [56, 57] distributed training strategy. Notably, the Qwen2.5-Coder-7B model underwent FP32 single-precision training to maximize numerical stability, achieving leading performance under the same parameters with 87.80 pass@1 about HumanEval, 67.0 pass@1 on MBPP, and 27.2 pass@1 on LiveCodeBench.

4.2.1 Phase task alignment

The alignment between the multi-task joint fine-tuning phase and RL phase primarily focuses on the PoT (Pseudocode-oriented Thinking) task and code completion tasks. Leverage the Qwen series base models, which incorporate intermediate FIM tasks during pre-training. By aligning the task forms in the subsequent stages, the framework ensures consistent task distributions across downstream phases, so that the model can obtain a better learning effect on downstream tasks. This alignment is reinforced through hierarchical task design across training stages. Early stages introduce foundational skills (e.g., syntax-aware code completion), while later stages emphasize complex logic synthesis (e.g., pseudocode-guided code generation). Intermediate tasks bridge these phases, narrowing the distribution gap between training objectives and target tasks to reduce generalization errors. For the newly introduced pseudocode abstract syntactic feature, task alignment is rigorously maintained across all phases. For example, PoT tasks in fine-tuning align with RL's preference optimization for pseudocode consistency, ensuring the model deeply internalizes this feature. This systematic alignment enables the model to incrementally master both basic programming constructs and advanced logical inference, culminating in robust code generation capabilities.

As shown in Table 3, the introduction of optimizations at each stage improved the model's performance on code generation and completion tasks, especially on the

Table 3 Ablation experiments at each stage based on the Qwen2.5-Coder-7B-Instruct model. The Qwen2.5-Coder-7B-Instruct model is fine-tuned with full parameters on a multi-task dataset consisting of six types of tasks, followed by SimPO RL using a mixed-task preference dataset. Finally, the resulting model is evaluated through inference using Self-Infilling, with benchmark testing on HumanEval and MBPP at each stage

Methods	HumanEval pass@1	MBPP pass@1
Qwen2.5-Coder-7B-Instruct	84.15	65.4
+ Full_Multi_SFT	84.76	65.0
+ Mixed_SimPO	87.80	67.0
+ Self-Infilling	88.15	67.6
Qwen2.5-Coder-7B-Instruct+Mixed_SimPO	84.42	65.6
Qwen2.5-Coder-7B-Instruct+Self-Infilling	85.27	65.8

HumanEval benchmark, which is more aligned with code completion tasks. After the multi-task introduction, the improvement over the original model was minimal, particularly after incorporating the PoT task and code completion task, where the HumanEval metric still showed no significant improvement under task alignment. However, after RL, the model showed a significant improvement. Subsequent Self-Infilling inference further maintained the model's performance. This is because the PoT task introduced diverse abstract task forms, enhancing the model's generalization ability in different contexts and making it more aligned with the actual requirements of code tasks. When only SimPO RL is applied to the baseline model, it only produces a moderate+0.27 gain (84.15 → 84.42) compared to the baseline. Similarly, when only Self-Infilling inference is applied to the baseline model, HumanEval produces a moderate+1.12 gain (84.15 → 85.27), while MBPP produces a moderate+0.4 gain (65.4 → 65.8), indicating that Self-Infilling provides additional robustness improvement. Each optimization stage brings incremental benefits, but the combined effect of multi-task learning and RL alignment is the main driving force, resulting in the largest jump, with HumanEval reaching 87.80 (+3.65). Due to the limitations of greedy decoding, the model's divergence ability could not be fully demonstrated. To address this, Sect. 4.2.2 validates performance using HumanEval+ (extended test cases) and the pass@10 metric, which evaluates correctness across multiple sampling attempts, providing deeper insights into the model's robustness and adaptability.

In the RL phase, task alignment was achieved with the PoT task and code completion task based on the multi-task joint fine-tuning phase. As shown in Table 4, when the PoT task and code completion task were sequentially removed during the RL phase, the model experienced a decline in the pass@1 metric on the HumanEval benchmark and the pass@1 metric on the MBPP benchmark. Without task alignment, the pass@1 metric on the MBPP benchmark even dropped lower than that of the model from the first stage in Table 3. This was due to the PoT task being more aligned with the MBPP benchmark, and the model experienced some forgetting of the PoT task during the RL phase. Similarly, when comparing the pass@1 metric based on the HumanEval benchmark across two stages, after removing the code completion task, the model also exhibited forgetting of the code completion task in the RL phase, highlighting the necessity of task alignment at each stage. Furthermore, as shown in Table 4, the decline in LiveCodeBench performance further confirms that the absence of key aligned tasks in the RL phase weakens the model's ability to generalize to real-world coding scenarios. This emphasizes the critical role

Table 4 Ablation experiment on task alignment using the model aligned after multi-task joint fine-tuning and SimPO RL with Qwen2.5-Coder-7B-Instruct as the baseline model

Methods	HumanEval pass@1	MBPP pass@1	LiveCodeBench pass@1
Qwen2.5-Coder-7B-Instruct+Full+SimPO	87.80	67.0	27.2
- PoT	85.98	64.0	25.5
- Code_completion	82.93	64.0	24.4

of preserving task-specific signals, particularly PoT and code completion, to ensure robust downstream performance across diverse benchmarks.

4.2.2 Multi-task design and the significance of pseudocode tasks

Code tasks have characteristics similar to multi-modal tasks, as they inherently involve two different modalities: programming languages and natural languages. Additionally, their usage scenarios vary, such as natural language generating code and code completion. Therefore, in multi-task design, code tasks need to align with various scenarios, and the introduction of pseudocode tasks can better integrate the modality features of both languages. Meanwhile, the HumanEval, MBPP, and LiveCodeBench focus on different aspects of coding abilities: the HumanEval benchmark leans toward testing completion abilities, while the MBPP benchmark evaluates natural language code generation instructions. Compared to HumanEval and MBPP, LiveCodeBench is more inclined toward comprehensive coding ability evaluation in real-world scenarios. These benchmarks align with various single-task and multi-task configurations, as shown in Table 5.

As shown in Table 5, in the comparison with the Qwen2.5-Coder-7B-Instruct baseline model, it is evident that the performance of various single-task fine-tunings did not yield good results. However, the introduction of pseudocode and PoT clearly demonstrated advantages over other single tasks, showing improvements in both benchmarks. The PoT paradigm reduces parameter conflicts between sub-tasks (such as code generation and completion) by decoupling high-level inference (pseudocode) from low-level syntax, helping the model understand code logic and higher-level syntactic structures, thus enhancing the accuracy and usability of the generated code. Additionally, the consistent uplift in LiveCodeBench pass@1 cross-single-task and multi-task settings highlights the generalization benefits of incorporating pseudocode alignment into multi-task training.

During the training process, it was observed that for single tasks like Text2Code and Code Completion, the model exhibited signs of early convergence. As shown in Fig. 8, the model tends to overfit quickly during single-task fine-tuning, triggering early stopping and leading to insufficient training. The multi-task joint fine-tuning model, by optimizing multiple tasks simultaneously (e.g., code completion, natural

Table 5 Comparison experiment of pass@1 metrics for single-task and multi-task models on the HumanEval, MBPP, and LiveCodeBench benchmarks

Methods	HumanEval pass@1	MBPP pass@1	LiveCodeBench pass@1
Qwen2.5-Coder-7B-Instruct	84.15	65.4	25.4
SFT-S-CODECOMPLETION	78.05	63.8	23.5
SFT-S-TEXT2CODE	82.32	67.6	25.4
SFT-S-PSEUDOCODE	84.15	67.2	25.7
Multi-3TASKS	82.32	64.2	23.7
Multi-6TASKS	84.76	65.0	25.9

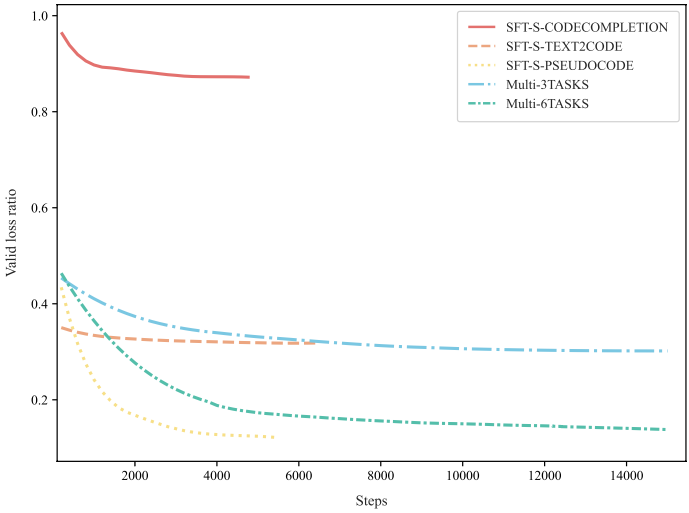

Fig. 8 Training verification loss curve of each fine-tuning model. Qwen2.5-Coder-7B-Instruct performs full-parameter fine-tuning on both single-task and multi-task datasets, with one validation after every 100 training steps. If there is no decrease in loss after five consecutive validations, an early stop is triggered

language code generation), enhanced data diversity and improved the model's generalization ability. The feature sharing and regularization effects between different tasks delayed overfitting, allowing the model to continue improving performance with more training steps.

The Text2Code single-task model performed well on the MBPP benchmark, which leans toward instruction-based testing, while the Code Completion single-task model performed relatively better on the HumanEval benchmark, which focuses on code completion testing. Both models demonstrated a bias toward their respective tasks, with their overall code generation capabilities slightly declining compared to the baseline model. After the introduction of multi-tasking, it was observed during training that multi-task learning served as a regularization technique, improving the model's ability to learn continuously while sharing representations across tasks. This led to faster convergence and a more balanced learning outcome across the tasks.

Combining Table 3 and Fig. 9, the introduction of pseudocode and PoT tasks through high-level abstract features builds more robust semantic representations, which help the model transfer more efficiently to code generation tasks. Pseudocode, by abstractly describing the syntactic structure of task intent, provides stronger logical clues, structured information, and contextual constraints. This explicit, structured learning reduces the possibility of the model generating invalid code. However, its logic and structure-based learning does not always directly reflect in the results of code generation, such as the lack of a significant improvement in the pass@1 metric after multi-task joint fine-tuning. It guides the model to learn process-related features of code generation, such as function logic, variable dependencies, and syntactic consistency. The instruction fine-tuning model shows significant improvement

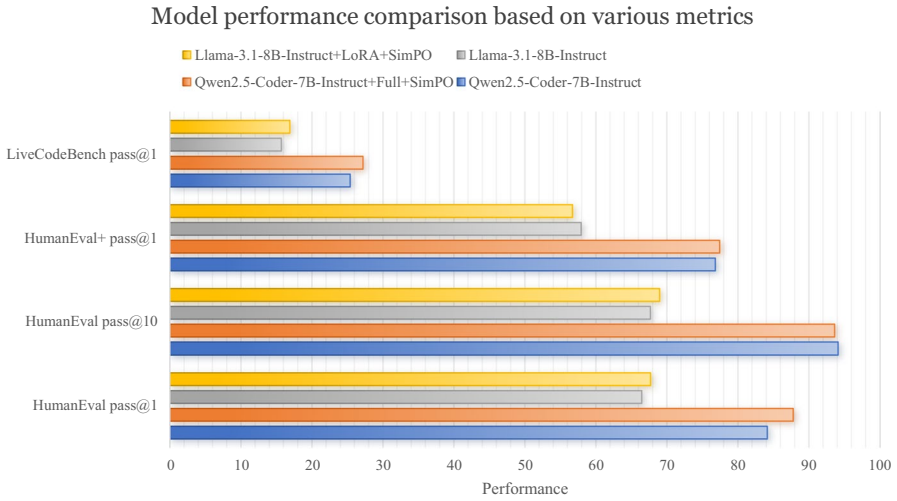

Fig. 9 Comparison experiment of various benchmark indicators after the second stage of the selected model. The HumanEval pass@10 is evaluated using a sampling strategy

after RL optimization of preferences. This is because pseudocode plays an important role in optimizing decision-making, helping the model focus on divergent sampling choices. As shown in Fig. 9, the Qwen model's pass@10 metric on the HumanEval benchmark, after RL, shows no significant difference from the baseline model, but the pass@1 metric significantly improves. Furthermore, on the expanded and more complex HumanEval+ benchmark, the RL model continues to outperform the baseline model. Additionally, for both LoRA fine-tuning and full-parameter fine-tuning, full-parameter fine-tuning in SimPO-based RL shows more significant results than LoRA fine-tuning.

4.3 Comparison of generation effects for different scenarios and complexity

To assess the proposed UCP system's performance across different languages and problem difficulties, we conducted additional experiments comparing its outputs on multilingual and algorithmically complex tasks. Specifically, we evaluated the Qwen2.5-Coder-7B-Instruct+Full+SimPO+Self-Infilling model on function-completion problems from HumanEval-X (in Python, C++, and Java) as well as on challenging algorithmic problems from the MBPP and LiveCodeBench benchmarks. Figures 10, 11, and 12 present representative example solutions generated for HumanEval-X problems in Python, C++, and Java, respectively. These examples illustrate the model's ability to adapt its reasoning to different programming paradigms (command-oriented scripting in Python, low-level control in C++, and object-oriented/functional constructs in Java). Figures 13 and 14 show examples from the MBPP and LiveCodeBench benchmarks in Python, highlighting the system's reasoning on dynamic programming and recursive, memoized solutions for

Prompt:

<|role_start|>human<|role_end|># language: **Python**
Complete the following python code:
def tri(n):
"""Everyone knows Fibonacci sequence, it was studied deeply by mathematicians in the last couple centuries. However, what people don't know is Tribonacci sequence.
Tribonacci sequence is defined by the recurrence:
tri(1) = 3
tri(n) = 1 + n / 2, if n is even.
tri(n) = tri(n - 1) + tri(n - 2) + tri(n + 1), if n is odd.
For example:
tri(2) = 1 + (2 / 2) = 2
tri(4) = 3
tri(3) = tri(2) + tri(1) + tri(4) = 2 + 3 + 3 = 8
You are given a non-negative integer number n, you have to a return a list of the first n + 1 numbers of the Tribonacci sequence.
Examples:
tri(3) = [1, 3, 2, 8]
"""
<|role_start|>bot<|role_end|>

Qwen2.5-Coder-7B-Instruct+Full+SimPO+Self-Infilling:

Here's how you can complete the Python code for generating the Tribonacci sequence based on the provided recurrence relations:

```python
def tri(n):
    # Initialize the base cases
    if n == 0:
        return [1]
    tribonacci = [1, 3]  # First two elements of the sequence
    for i in range(2, n + 1):
        if i % 2 == 0:  # If n is even
            tribonacci.append(1 + i / 2)
        else:  # If n is odd
            tribonacci.append(tribonacci[i - 1] + tribonacci[i - 2] + (i + 3) / 2)
    return tribonacci

# Test the function
print(tri(3))  # Output: [1, 3, 2, 8]
```

Fig. 10 Example of the UCP model solving a HumanEval-X Python problem (Tribonacci sequence)

hard problems. In all cases, the solutions are correct (all test cases pass) and demonstrate a clear logical structure.

The generated implementation covers the base case as well as both even and odd recursive branches, yielding outputs that match the expected example and pass all test cases (100%). The approach straightforwardly follows the recursive specification: it incrementally implements the relationship described in the problem and names variables (e.g., for sequence values) clearly. The control flow and branching logic are simple and transparent, reflecting a clear understanding of the problem's recursive structure.

This solution correctly determines whether all numbers in an input list consist solely of odd digits and then returns those numbers in ascending order. It achieves a 100% pass rate on the test suite. The code uses explicit loops and the C++ standard library sort function, with intuitive variable names (such as allOdd, temp, digit) that make the logic clear. The overall structure is straightforward and easy to follow. Notably, while the solution is effective, it does not employ more advanced C++ algorithms (for example, it could have used std::all_of with a custom predicate), which suggests a trade-off between clarity and use of higher-level abstractions.

The generated code counts the frequency of each element in a list and then filters elements that appear only once, using modern Java constructs. It makes full use of HashMap.getOrDefault for frequency counting and the Stream API to filter and collect results, reflecting contemporary Java programming practices. The

Prompt:

```
<|role_start|>human<|role_end|># language: CPP
/*
Given a vector of positive integers x. return a sorted vector of all
elements that hasn't any even digit.
Note: Returned vector should be sorted in increasing order.
For example:
>>> unique_digits({15, 33, 1422, 1})
{1, 15, 33}
>>> unique_digits({152, 323, 1422, 10})
{}
*/
Complete the following python code:
#include<stdio.h>
#include<vector>
#include<algorithm>
using namespace std;
vector<int> unique_digits(vector<int> x){
<|role_start|>bot<|role_end|>
```

Qwen2.5-Coder-7B-Instruct+Full+SimPO+Self-Infilling:

Here's how you can complete the C++ code to filter and sort numbers composed only of odd digits:

```cpp
vector<int> out;  // Will hold numbers composed only of odd digits
    for (int num : x) {
        bool allOdd = true;  // Flag to track if current number has only odd digits
        if (num == 0) {
            allOdd = false;
        }
        int temp = num;
        while (temp > 0 && allOdd) {
            int digit = temp % 10;      // Extract last digit
            if (digit % 2 == 0) {
                allOdd = false;         // Found an even digit → reject
            }
            temp /= 10;                 // Drop the last digit
        }
        if (allOdd) {
            out.push_back(num);      // If all digits were odd, add to output
        }
    }
    sort(out.begin(), out.end());  // Sort the filtered numbers in ascending order
    return out;
}
```

Fig. 11 Example of the UCP model solving a HumanEval-X C++ problem

solution passes all test cases (100%), and the code is concise and easy to read. Its use of streams and functional-style operations demonstrates that the model can adapt to object-oriented and functional paradigms in Java while still producing correct, idiomatic code.

The task is algorithmic and requires finding the longest subsequence in which adjacent elements differ by exactly one. The model's solution applies a classic dynamic programming strategy: it maintains an array diff and uses two nested loops to update diff[i] = max(diff[i], diff[j] + 1) whenever the absolute difference abs(arr[i] - arr[j]) == 1. All test assertions pass (100%), indicating correct implementation of the algorithm. The nested loop structure and conditional check are clear and mirror standard DP patterns. Variable names such as diff and res align with common dynamic programming conventions. The code even handles edge cases gracefully (for example, returning correct results when the input list is empty). Overall, the implementation is concise and well commented, making the core DP logic easy to follow.

Prompt:

<|role_start|>human<|role_end|># language: **Java**
Complete the following python code:
import java.util.*;
import java.lang.*;
import java.util.stream.Collectors;
class Solution {
/*
From a list of integers, remove all elements that occur more than once.
Keep order of elements left the same as in the input.
>>> removeDuplicates(Array.asList(1, 2, 3, 2, 4))
[1, 3, 4]
*/
public List<Integer> removeDuplicates(List<Integer> numbers) {
<|role_start|>bot<|role_end|>

Qwen2.5-Coder-7B-Instruct+Full+SimPO+Self-Infilling:

Here's how you can complete the Java code to filter out non-unique elements from a list, keeping only the ones that appear exactly once:

```java
Map<Integer, Integer> counts = new HashMap<>();
      for (int num : numbers) {
            counts.put(num, counts.getOrDefault(num, 0) + 1);
      }
// Stream the original list, keep only those with count == 1, collect to a list
return numbers.stream()
              .filter(num -> counts.get(num) == 1)  // include only unique elements
              .collect(Collectors.toList());
    }
```

Fig. 12 Example of the UCP model solving a HumanEval-X Java problem

Prompt:

<|role_start|>human<|role_end|># language: **Python**
You are an expert Python programmer, and here is your task: Write a function to find the maximum length of the subsequence with difference between adjacent elements for the given array. Your code should pass these tests:

assert max_len_sub([2, 5, 6, 3, 7, 6, 5, 8], 8) == 5
assert max_len_sub([-2, -1, 5, -1, 4, 0, 3], 7) == 4
assert max_len_sub([9, 11, 13, 15, 18], 5) == 1
<|role_start|>bot<|role_end|>

Qwen2.5-Coder-7B-Instruct+Full+SimPO+Self-Infilling:

```python
def max_len_sub(arr, n):
    if n == 0:      # Edge case: empty array
        return 0
    diff = [1] * n  # subseq lengths
    res = 1         # res tracks the overall maximum
    for i in range(1, n):
        for j in range(i):
            if abs(arr[i] - arr[j]) == 1:
                diff[i] = max(diff[i], diff[j] + 1)
        res = max(res, diff[i])    # Update global maximum if needed
    return res
```

Fig. 13 Reasoning example of the UCP model solving an MBPP (Python) benchmark problem

In this complex recursive task, the model generates helper functions that use memoization (@cache) and explicitly control the recursion state. The solution controls upper and lower bounds using a tight parameter and accumulates a running sum (cur_sum) while pruning branches that exceed the bound. It correctly covers all necessary boundary conditions (including leading zeros and transitions between tight and loose states) and implements the required modular arithmetic. The implementation passes all test cases (100%), demonstrating full coverage of constraints. The use of @cache for memoization keeps the code concise, and the choice of parameter names (s, tight, cur_sum) intuitively reflects their roles in the

Prompt:

You are given two numeric strings num1 and num2 and two integers max_sum and min_sum. We denote an integer x to be good if:

- num1 ≤ x ≤ num2
- min_sum ≤ digit_sum(x) ≤ max_sum.

Return the number of good integers. Since the answer may be large, return it modulo $10^9 + 7$.
Note that digit_sum(x) denotes the sum of the digits of x.

Example 1:

Input: num1 = "1", num2 = "12", min_num = 1, max_num = 8
Output: 11
Explanation: There are 11 integers whose sum of digits lies between 1 and 8 are 1,2,3,4,5,6,7,8,10,11, and 12. Thus, we return 11.

Example 2:

Input: num1 = "1", num2 = "5", min_num = 1, max_num = 5
Output: 5
Explanation: The 5 integers whose sum of digits lies between 1 and 5 are 1,2,3,4, and 5. Thus, we return 5.

Constraints:

- 1 ≤ num1 ≤ num2 ≤ 10^{22}
- 1 ≤ min_sum ≤ max_sum ≤ 400

Qwen2.5-Coder-7B-Instruct+Full+SimPO+Self-Infilling:

```python
from functools import cache

class Solution:
    def count(self, num1: str, num2: str, min_sum: int, max_sum: int) -> int:
        mod = 10**9 + 7

        @cache
        def helper(s, tight, cur_sum):
            if cur_sum > max_sum:
                return 0
            if len(s) == 0:
                return 1 if cur_sum >= min_sum else 0
            ans = 0
            ub = int(s[0]) if tight else 9
            for d in range(ub + 1):
                ans += helper(s[1:], tight and d == ub, cur_sum + d)
                ans %= mod
            return ans
        return (helper(num2, True, 0) - helper(str(int(num1) - 1), True, 0) + mod) %
mod
```

Fig. 14 Reasoning example of the UCP model solving a hard LiveCodeBench (LeetCode) problem in Python

state. The code logic is clearly layered: it first handles pruning of the current sum, then recursively accumulates results, and finally applies modulo operations at the end of the computation. This structure is consistent with Pythonic practices and shows that the model can handle complex, hard-level algorithmic reasoning. (The only minor improvement could be to encapsulate the string-to-integer boundary transformation in a separate function to enhance maintainability, but this does not affect correctness.)

In summary, the examples in Figs. 10, 11, 12, 13 and 14 demonstrate that the proposed UCP model consistently generates correct, well-structured solutions across diverse scenarios. In multilingual code generation (Figs. 10, 11 and 12), it adapts to language-specific idioms and paradigms while correctly implementing the required logic. In algorithmically complex problems (Figs. 13 and 14), it applies appropriate reasoning strategies (dynamic programming, recursion with memoization) and handles edge cases effectively. These results indicate strong

reasoning capability and adaptability of the model across different programming languages and levels of problem complexity.

5 Conclusion

This paper presents a process framework aimed at further improving the code capabilities of various foundational models and instruction-tuned models. By introducing abstract syntactic features of pseudocode through the PoT task and the self-constructed multi-task dataset, the framework aligns tasks across two stages of multi-task joint fine-tuning and RL and further aligns three stages in the inference phase through a cycle of Self-Infilling in a fill-in-the-blank task form. Using the Qwen2.5-Coder-7B-Instruct model as the baseline, it achieves leading performance with a similar parameter scale on the HumanEval benchmark, effectively demonstrating the effectiveness of introducing pseudocode and related process designs in improving the model's code performance. Unlike methods that focus on algorithmic structures or stacking computational power and model parameters, the process framework provides an efficient, plug-and-play solution under limited computational resources for any open-source LLMs.

Limitations and Future Directions. Meanwhile, our work complements the direction of commercial assistants by demonstrating how structured, interpretable reasoning (via PoT) can be incorporated into open-source code models, offering not only competitive performance but also enhanced controllability and interpretability that are currently underexplored in production-grade tools. Therefore, there are a number of interesting directions for future work on UCP, some of which we outline below:

- When UCP uses open-source models, there is still a gap between it and GPT-o4-mini models that have comprehensive capabilities in complex code generation tasks. We believe that when facing complex code generation problems involving complex fields such as mathematics and language, the comprehensive capabilities of the model are of great help for code generation. Therefore, how to maintain the comprehensive ability of the model while improving the code capability of the model will become the focus of the next step of work.
- Cyclic Self-Infilling reasoning incurs additional computational overhead. While we have optimized cyclic Self-Infilling through early stopping, dynamic cycling, and partial or conditional decoding, there remains room for further optimization. Future work may focus on enhancing key-value caching and cross-iteration reuse to improve efficiency.
- In the future, we will also explore the tight integration of LLM inference and editors, as well as the enterprise functions of these tools (privacy, context indexing, policy execution), on how to rely on agents in the IDE to handle context and interface details, rather than manually formalizing prompts into universal LLM, and explore ways to more closely integrate AI-assisted tools such as Tabnine and Cursor with LLMs.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s11227-025-07487-1.

References

1. Achiam J, Adler S, Agarwal S et al (2023) Gpt-4 technical report. arXiv preprint arXiv: 2303.08774
2. Feng Z, Guo D, Tang D et al (2020) Codebert: A pre-trained model for programming and natural languages. Findings of the Association for Computational Linguistics: EMNLP 2020:1536–1547
3. Wang Y, Wang W, Joty S et al (2021) Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp 8696–8708
4. Li Y, Choi D, Chung J et al (2022) Competition-level code generation with alphacode. Science 378(6624):1092–1097
5. Chen M, Tworek J, Jun H et al (2021) Evaluating large language models trained on code. arXiv preprint arXiv: 2107.03374
6. Allal LB, Li R, Kocetkov D et al (2023) Santacoder: don't reach for the stars! arXiv preprint arXiv: 2301.03988
7. Roziere B, Gehring J, Gloeckle F et al (2023) Code llama: Open foundation models for code. arXiv preprint arXiv: 2308.12950
8. Hu EJ, Shen Y, Wallis P et al (2022) Lora: Low-rank adaptation of large language models. ICLR 1(2):3
9. Dettmers T, Pagnoni A, Holtzman A et al (2023) Qlora: Efficient finetuning of quantized llms. Adv Neural Inf Process Syst 36:10088–10115
10. Lewis P, Perez E, Piktus A et al (2020) Retrieval-augmented generation for knowledge-intensive nlp tasks. Adv Neural Inf Process Syst 33:9459–9474
11. Wei J, Wang X, Schuurmans D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 35:24824–24837
12. Li J, Li G, Li Y et al (2025) Structured chain-of-thought prompting for code generation. ACM Transactions on Software Engineering and Methodology 34(2):1–23
13. GLM T, Zeng A, Xu B et al (2024) Chatglm: A family of large language models from glm-130b to glm-4 all tools. arXiv preprint arXiv: 2406.12793
14. Meng Y, Xia M, Chen D (2025) Simpo: Simple preference optimization with a reference-free reward. Adv Neural Inf Process Syst 37:124198–124235
15. Bai J, Bai S, Chu Y et al (2023) Qwen technical report. arXiv preprint arXiv: 2309.16609
16. Raffel C, Shazeer N, Roberts A et al (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res 21(140):1–67
17. Zheng L, Yuan J, Zhang Z et al (2023) Self-infilling code generation. arXiv preprint arXiv: 2311.17972
18. Hui B, Yang J, Cui Z et al (2024) Qwen2. 5-coder technical report. arXiv preprint arXiv: 2409.12186
19. Austin J, Odena A, Nye M et al (2021) Program synthesis with large language models. arXiv preprint arXiv: 2108.07732
20. Jain N, Han K, Gu A et al (2024) Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv: 2403.07974
21. Donvir A, Panyam S, Paliwal G et al (2024) The role of generative ai tools in application development: a comprehensive review of current technologies and practices. In: 2024 International Conference on Engineering Management of Communication and Technology (EMCTECH), IEEE, pp 1–9
22. Thoppilan R, De Freitas D, Hall J et al (2022) Lamda: Language models for dialog applications. arXiv preprint arXiv: 2201.08239
23. Anil R, Dai AM, Firat O et al (2023) Palm 2 technical report. arXiv preprint arXiv: 2305.10403
24. Hurst A, Lerer A, Goucher AP et al (2024) Gpt-4o system card. arXiv preprint arXiv: 2410.21276
25. Christopoulou F, Lampouras G, Gritta M et al (2022) Pangu-coder: Program synthesis with function-level language modeling. arXiv preprint arXiv: 2207.11280
26. Zan D, Chen B, Yang D et al (2022) Cert: continual pre-training on sketches for library-oriented code generation. arXiv preprint arXiv: 2206.06888

27. Fried D, Aghajanyan A, Lin J et al (2022) Incoder: a generative model for code infilling and synthesis. arXiv preprint arXiv: 2204.05999
28. Li R, Allal LB, Zi Y et al (2023) Starcoder: may the source be with you! arXiv preprint arXiv: 2305.06161
29. Luo Z, Xu C, Zhao P et al (2023) Wizardcoder: empowering code large language models with evol-instruct. arXiv preprint arXiv: 2306.08568
30. Wang Y, Le H, Gotmare A et al (2023) Codet5+: open code large language models for code understanding and generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp 1069–1088
31. Liu B, Chen C, Gong Z et al (2024) Mftcoder: boosting code llms with multitask fine-tuning. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp 5430–5441
32. Shen B, Zhang J, Chen T et al (2023) Pangu-coder2: boosting large language models for code with ranking feedback. arXiv preprint arXiv: 2307.14936
33. Xu C, Sun Q, Zheng K et al (2024) Wizardlm: empowering large pre-trained language models to follow complex instructions. In: The Twelfth International Conference on Learning Representations
34. Le H, Wang Y, Gotmare AD et al (2022) Coderl: mastering code generation through pretrained models and deep reinforcement learning. Adv Neural Inf Process Syst 35:21314–21328
35. Wang X, Wang Y, Wan Y et al (2022) Compilable neural code generation with compiler feedback. Findings of the Association for Computational Linguistics: ACL 2022:9–19
36. Shojaee P, Jain A, Tipirneni S et al (2023) Execution-based code generation using deep reinforcement learning. arXiv preprint arXiv: 2301.13816
37. Schulman J, Wolski F, Dhariwal P et al (2017) Proximal policy optimization algorithms. arXiv preprint arXiv: 1707.06347
38. Liu J, Zhu Y, Xiao K et al (2023) Rltf: reinforcement learning from unit test feedback. arXiv preprint arXiv: 2307.04349
39. Guo D, Ren S, Lu S et al (2020) Graphcodebert: pre-training code representations with data flow. arXiv preprint arXiv: 2009.08366
40. Jiang X, Zheng Z, Lyu C et al (2021) Treebert: a tree-based pre-trained model for programming language. In: Uncertainty in Artificial Intelligence, PMLR, pp 54–63
41. Zha Y, Yang Y, Li R et al (2023) Text alignment is an efficient unified model for massive nlp tasks. Adv Neural Inf Process Syst 36:77942–77968
42. Rafailov R, Sharma A, Mitchell E et al (2023) Direct preference optimization: Your language model is secretly a reward model. Adv Neural Inf Process Syst 36:53728–53741
43. Mo R, Wang D, Zhan W et al (2025) Assessing and analyzing the correctness of github copilot's code suggestions. ACM Transactions on Software Engineering and Methodology. https://doi.org/10.1145/3715108
44. Yetiştiren B, Özsoy I, Ayerdem M et al (2023) Evaluating the code quality of ai-assisted code generation tools: an empirical study on github copilot, amazon codewhisperer, and chatgpt. arXiv preprint arXiv: 2304.10778
45. Solovyeva L, Weidmann S, Castor F (2025) Ai-powered, but power-hungry? energy efficiency of llm-generated code. arXiv preprint arXiv: 2502.02412
46. Vaswani A, Shazeer N, Parmar N et al (2017) Attention is all you need. Advances in neural information processing systems 30
47. Dubey A, Jauhri A, Pandey A et al (2024) The llama 3 herd of models. arXiv preprint arXiv: 2407.21783
48. Gong Z, Yu H, Liao C et al (2024) Coba: convergence balancer for multitask finetuning of large language models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp 8063–8077
49. Liu J, Xia CS, Wang Y et al (2023) Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. Adv Neural Inf Process Syst 36:21558–21572
50. Kwon W, Li Z, Zhuang S et al (2023) Efficient memory management for large language model serving with pagedattention. In: Proceedings of the 29th Symposium on Operating Systems Principles, pp 611–626
51. Yang A, Yang B, Hui B et al (2024) Qwen2 technical report. arXiv preprint arXiv: 2407.10671
52. Jiang AQ, Sablayrolles A, Mensch A et al (2023) Mistral 7b. arXiv preprint arXiv: 2310.06825
53. Mistral AI Team (2024) Un ministral, des ministraux. mistral Research Licensehttps://mistral.ai/fr/news/ministraux/,

54. Zhu Q, Guo D, Shao Z et al (2024) Deepseek-coder-v2: breaking the barrier of closed-source models in code intelligence. arXiv preprint arXiv: 2406.11931
55. Liu A, Feng B, Xue B et al (2024) Deepseek-v3 technical report. arXiv preprint arXiv: 2412.19437
56. Rasley J, Rajbhandari S, Ruwase O et al (2020) Deepspeed: system optimizations enable training deep learning models with over 100 billion parameters. In: Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining, pp 3505–3506
57. Rajbhandari S, Rasley J, Ruwase O et al (2020) Zero: memory optimizations toward training trillion parameter models. SC20: International Conference for High Performance Computing. Networking, Storage and Analysis, IEEE, pp 1–16

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Yongjun Wen[1,2] · Zhihao Cui[1,2] · Yihao Liu[3] · Zhao Zhang[1,2] · Jiake Zhou[1,2] · Lijun Tang[1,2]

✉ Lijun Tang
tanglj@csust.edu.cn

Yongjun Wen
micowen@csust.edu.cn

1 School of Physics & Electronic Science, Changsha University of Science & Technology, Changsha 410114, Hunan, China

2 Hunan Province Higher Education Key Laboratory of Modeling and Monitoring on the Near-Earth Electromagnetic Environments, Changsha 410114, Hunan, China

3 Cancer Research Institute, Central South University, Changsha 410083, Hunan, China